sustantiva

CON•SENTIDO

DESARROLLO DE APLICACIONES FRONT-END TRAINEE V2.0

Pruebas en Vue

- Pruebas Unitarias
- Características de las pruebas unitarias
- El Desarrollo Dirigido por Test (TDD), en qué consiste y qué importancia tiene
- Setup de herramientas con vue-cli
- El entorno de pruebas Vue Test Utils para Vue
- Herramientas para el testing unitario, ventajas y
- limitaciones de cada uno Jest, Mocha + Chai
- Utilización de objetos simulados (mocks y stubs) En qué consisten y por qué utilizarlos
- Pruebas end-to-end

.- Pruebas end-to-end
.- Características de las pruebas end to end, ventajas y limitaciones, diferencia con pruebas unitarias
.- Setup de herramientas con vue-cli
.- Herramientas para el testing end to end, ventajas y limitaciones de cada uno
.- Cypress
.- Nightwatch

Pruebas Unitarias

Un test unitario o “prueba unitaria” , es un tipo de test automatizado . Este test, se utiliza para comprobar que un método concreto del código de nuestra aplicación funciona correctamente

Los test deben ser:

- Rapidos.
- Aislados: No tienen que depender el orden de ejecución de los mismos.
- Repetibles: funciona siempre o falla siempre.
- Fáciles de entender y escribir.
- Completas: Deben cubrir la mayor cantidad de código.
- Profesionales: Mismo criterio que se aplica al código de producción.

Caracteristicas de las pruebas unitarias

La regla general para implementar un test unitario es que este debe:

- Ser simple de comprender
- Ejecutarse de forma rápida.
- Hacer comprobaciones acerca de una única unidad de código.

El Desarrollo Dirigido por Test (TDD), en qué consiste y qué importancia tiene?

Desarrollo guiado por pruebas de software, o Test-driven development (TDD) , es una técnica que se basa en un desarrollo de pruebas a través de las cuales debe pasar el código, si está bien implementado, se continúa creando código enfocado hacia la siguiente prueba, sinó, se configura hasta que pase la prueba y podamos ir a la siguiente fase. Involucra esencialmente la práctica de escribir las pruebas primero, y generalmente estas pruebas son pruebas unitarias.

El propósito del desarrollo guiado por pruebas es lograr un código limpio que funcione. La idea es que los requisitos sean traducidos a pruebas, de este modo, cuando las pruebas pasen se garantizará que el software cumple con los requisitos que se han establecido.
Su ciclo de desarrollo es:

- Elegimos una prueba.
- Sometemos el código a la prueba.
- Si da error, resolvemos el problema.
- Volvemos a realizar la prueba hasta que dé positivo y pasamos al siguiente test.
- Si no da error, pasamos a la siguiente prueba.

¿Cuáles son los Beneficios del Desarrollo Dirigido por Pruebas?

Además de los beneficios mencionados en la sección anterior, el TDD también le consigue:

Notificación temprana de errores.
Diagnóstico sencillo de los errores, ya que las pruebas identifican lo que ha fallado.
Lo que todo esto significa para su negocio, es:

Mejora su factor de bus, ya que el conocimiento no está solo en las cabezas y facilita la incorporación de nuevas contrataciones.
Reduce el coste de las mejoras. Mantener el código limpio es también la forma de minimizar el riesgo de complicaciones accidentales. Y eso significa que podrá mantener un ritmo constante en la entrega de valor.
Con la red de seguridad, los desarrolladores están más dispuestos a fusionar sus cambios y a incorporar los de otros desarrolladores. Y, lo harán más a menudo. Y entonces el desarrollo basado en el tronco y la integración, entrega y despliegue continuos pueden despegar realmente.

Disminuye el número de errores que se «escapan» a la producción y eso reduce los costes de soporte.

Setup de herramientas con vue-cli

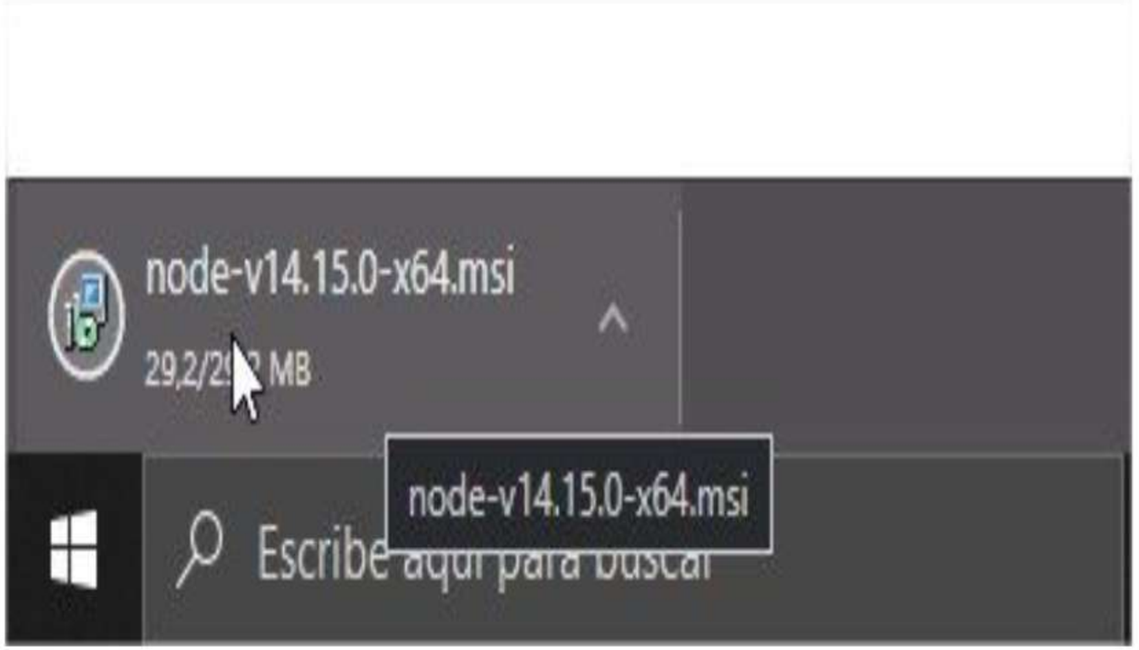
- Para instalar Vue.js 2 con Vue CLi tenemos que seguir los siguientes pasos.

- **Paso 1.** Tener instalado y actualizado NodeJS, puedes conseguirlo aquí https://nodejs.org/es/download/,

- En este post cubriremos la instalación de Node.js en Windows, a través del instalador que puedes descargar desde el sitio oficial de Node.js. El primer paso es https://nodejs.org/en/download/ y seleccionar el paquete correspondiente a tu sistema operativo, para esta guía se asume Windows, como aparece en el siguiente screenshot:

Pruebas en Vue

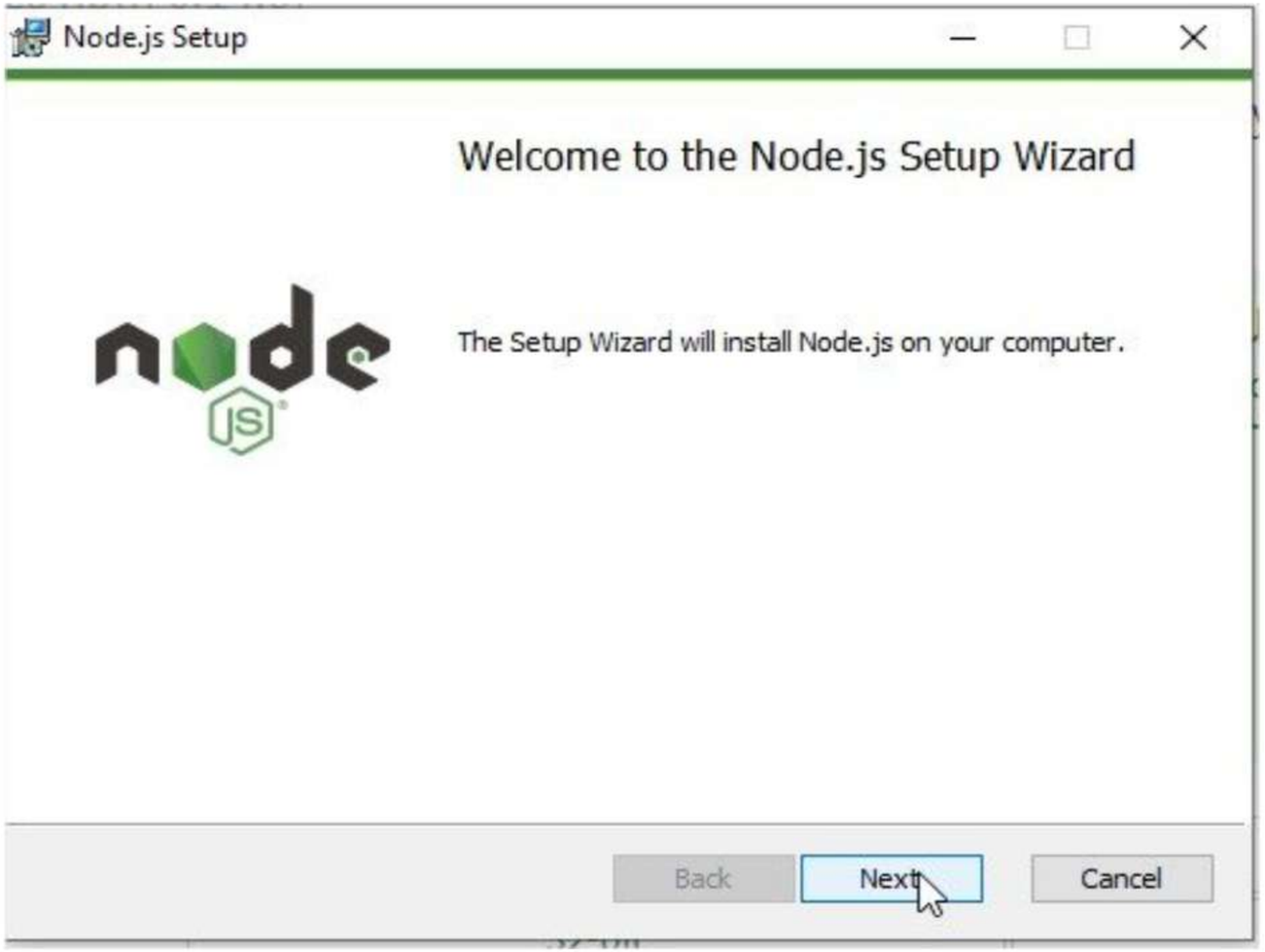
Al hacer clic verás cómo aparece un archivo en descarga en la parte inferior de tu navegador (si usas Chrome), puede aparecer de distinta manera en otros navegadores.



Pruebas en Vue

Instalar con el asistente de Node.js

Para iniciar la instalación y ejecutar el asistente de instalación, únicamente debes abrir el archivo recién descargado con extensión .msi. Este archivo abrirá una ventana como la que ves a continuación:

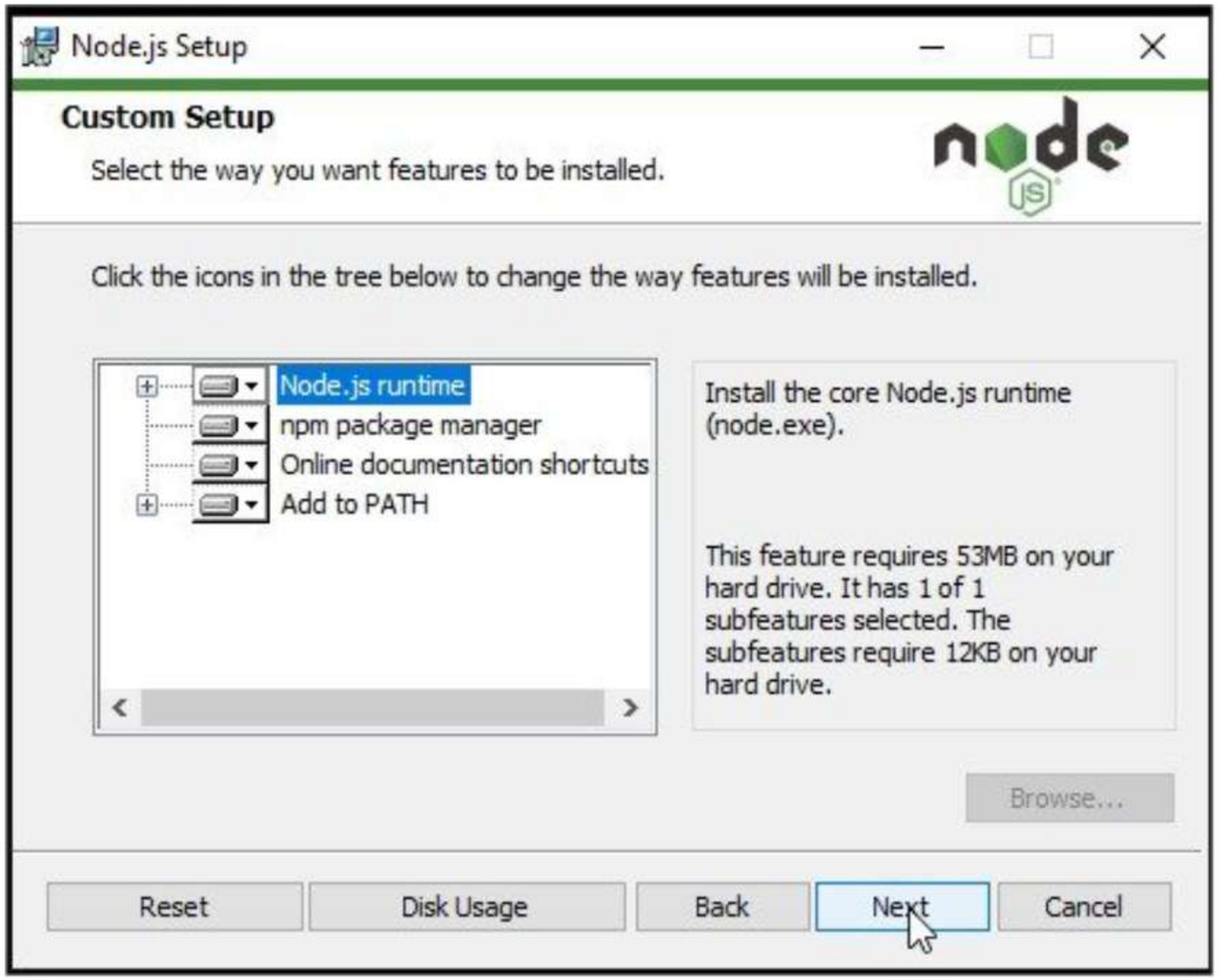


Pruebas en Vue

Esta es la pantalla de bienvenida, donde aparece un botón con la leyenda "Next" que debes seleccionar para iniciar el proceso de instalación, luego de eso, verás la pantalla para aceptar la licencia de uso de Node.js, en donde deberás activar la casilla con la leyenda "I accept the terms in the License Agreement" si aceptas los terminos y condiciones que aparecen en la misma ventana. Tal como se muestra a continuación:

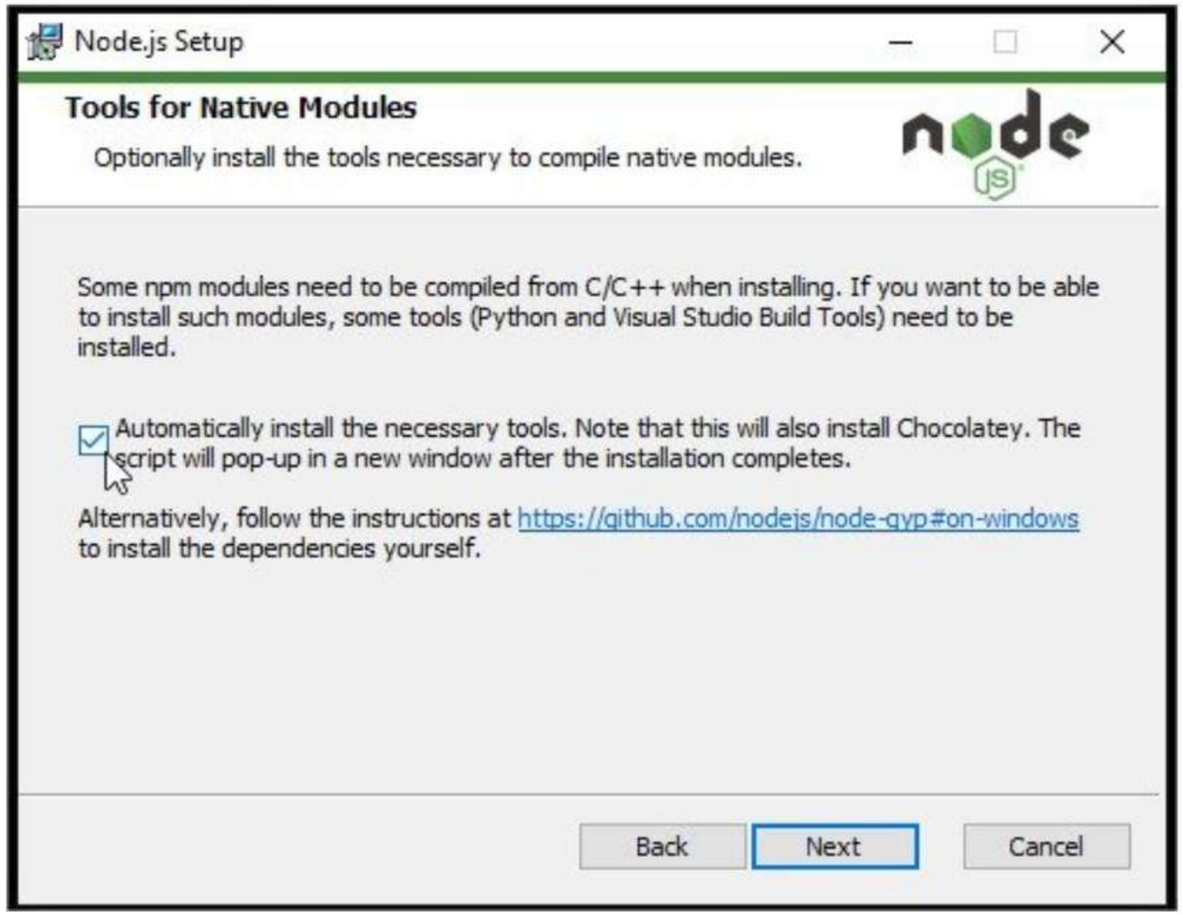
Pruebas en Vue

Luego de seleccionar la ruta, o si deseas dejar la que aparece por defecto, presiona el botón con la leyenda "Next". La siguiente pantalla te permite configurar la instalación de Node.js, a menos de que tengas experiencia con Node.js y sepas específicamente qué cosas necesitas y qué cosas no, te recomiendo dejar la instalación tal y como está, sin hacer configuraciones adicionales:



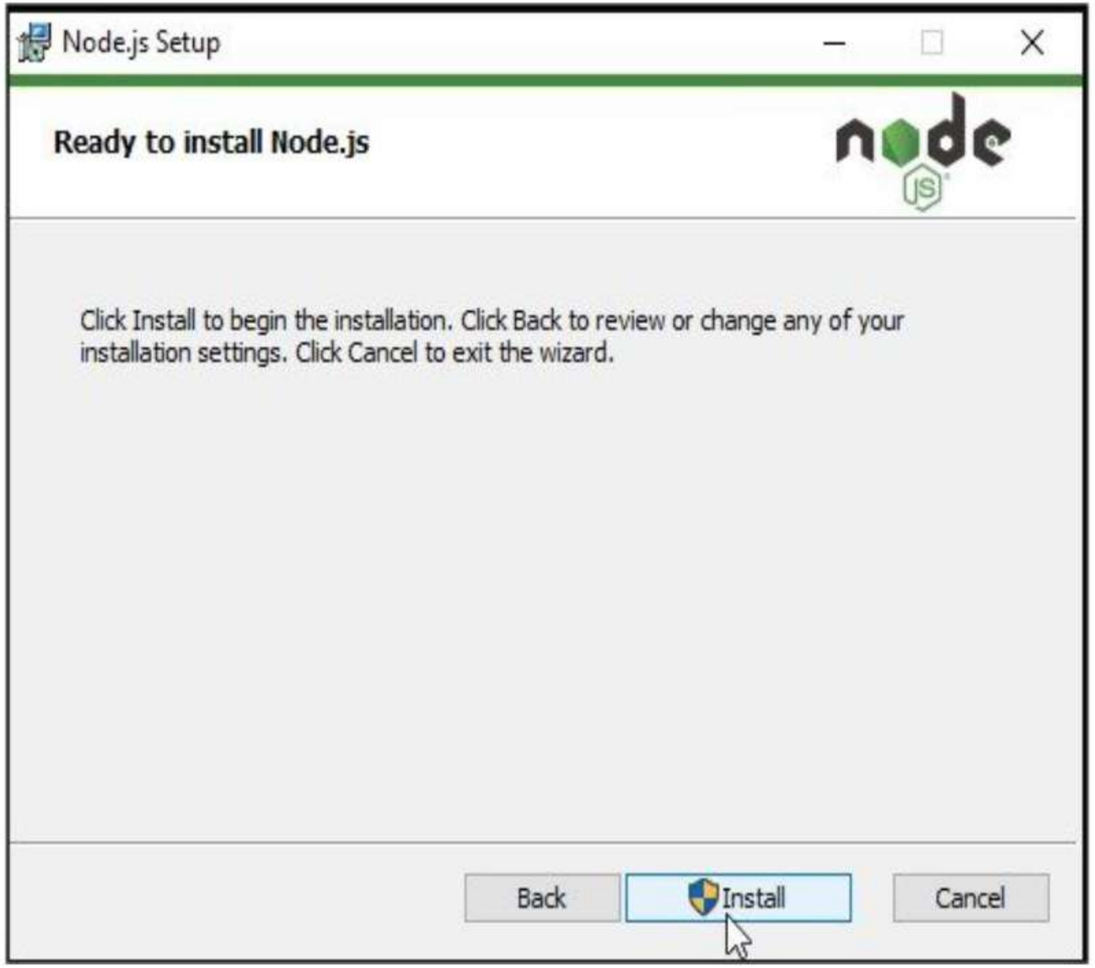
Pruebas en Vue

Asegúrate de presionar el botón con la leyenda "Next" para continuar. La siguiente pantalla solicita tu consentimiento para instalar herramientas adicionales a Node.js que son necesarias para que Node.js funcione, incluido el manejador de paquetes Chocolatey que puede servirte para instalar otras dependencias de desarrollo en tu computadora. Puedes consultar más información acerca de Chocolatey en https://chocolatey.org/. Asegúrate de habilitar la casilla con la leyenda "Automatically install the neccessary tools..."



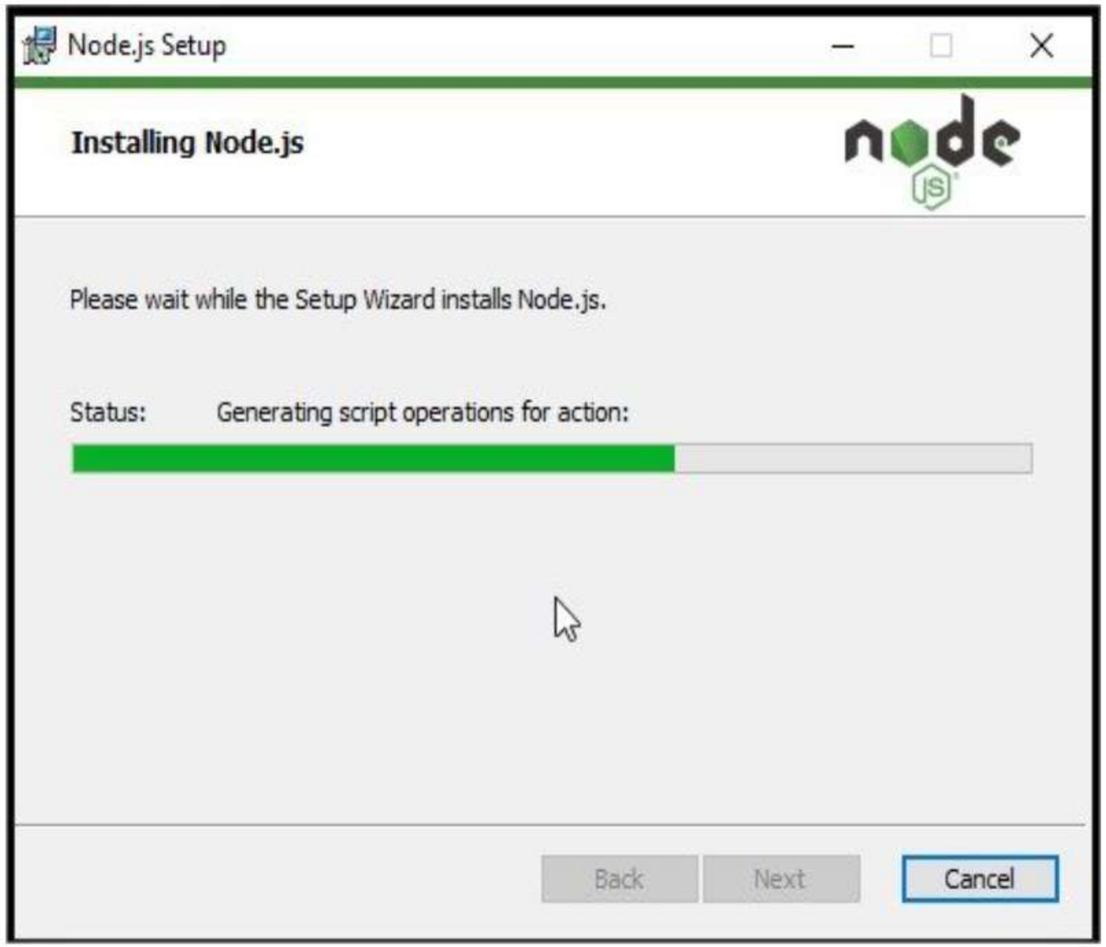
Pruebas en Vue

Para la última pantalla únicamente debemos presionar el botón con la leyenda "Install" para iniciar el proceso de instalación. Ten en cuenta que este botón puede solicitar que habilites permisos de Administrador.



Pruebas en Vue

La siguiente pantalla que verás contiene una barra de progreso con el status de la instalación, únicamente debes esperar a que esta barra se complete:

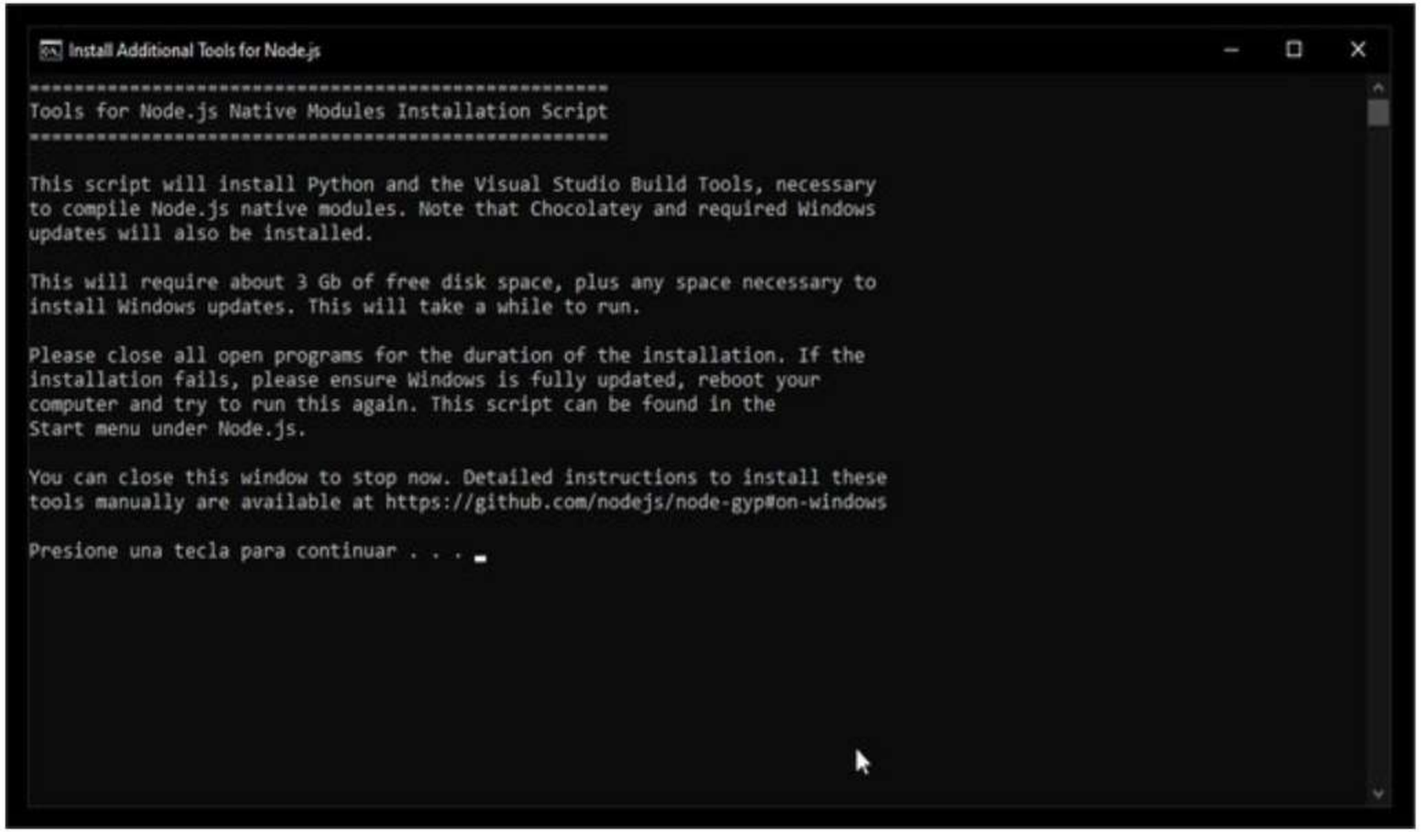


Pruebas en Vue

Por último verás una pantalla notificando que la instalación se ha completado exitosamente. Si la instalación no se ha completado exitosamente, deja un comentario en la sección de dudas. Haz clic en el botón con la leyenda "Finish" para continuar.

Pruebas en Vue

A continuación, automáticamente se abrirá una pantalla de la consola para la instalación de Chocolatey, tal como lo discutimos en un paso anterior:



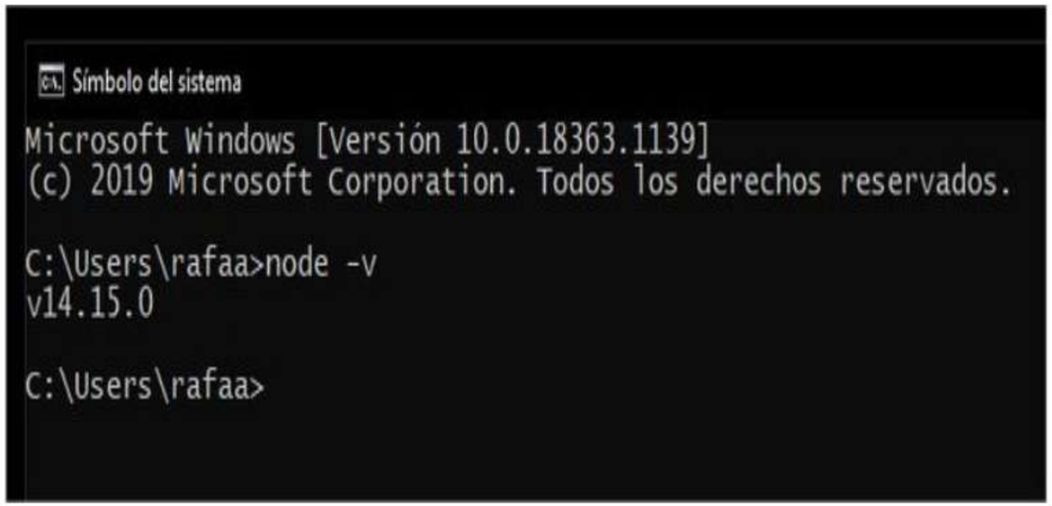
Probar tu instalación

Para probar tu instalación, puedes ejecutar los siguientes comandos en el CMD, el programa de Windows para la ejecución de comandos. Para abir el CMD presiona la tecla Windows, luego escribe CMD y haz clic en el programa con ese nombre.

A continuación escribe y ejecuta presionando Enter, el siguiente comando:

node –v

A continuación, debería aparecer la versión de Node.js que instalaste, tal como se muestra en la siguiente imagen:



Pruebas en Vue

Para corroborar que tienes todo lo necesario para continuar con el curso, escribe y ejecuta presionando Enter, el siguiente comando:

npm –v

A continuación verás la versión de NPM que tienes instalada, tal como se muestra en la imagen a continuación:

Con Node.js y npm instalados en tu computadora, ya puedes continuar con la instalación de vue/cli

```
C:\Users\rafaa>npm -v
6.14.8

C:\Users\rafaa>
```

Pruebas en Vue

Para instalar Vue CLI mediante npm debes ejecutar el siguiente comando

```
npm i -g @vue/cli @vue/cli-service-global
```

Para instalar Vue CLI mediante yarn debes ejecutar el siguiente comando:

```
yarn global add @vue/cli @vue/cli-service-global
```

Ahora que ya tenemos la aplicación Vue CLI instalada en nuestro sistema, ya podemos usar el comando vue.

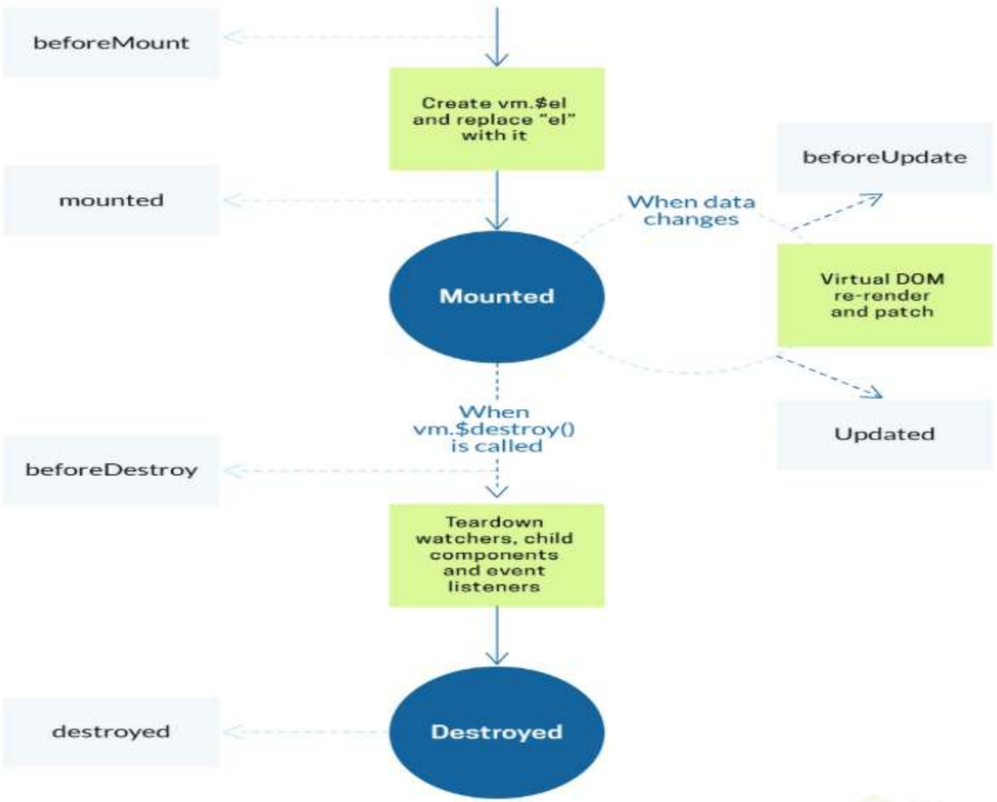
El entorno de pruebas vue Test Utils para Vue

Ejemplos de estas herramientas son el mismo Vue. Este framework está basado en unas APIs nativas de Webcomponents, pero son muy engorrosas de usar, ya que debemos conocer ampliamente la manera en la que se crean los elementos con la función CustomElements y aún hay navegadores que no las soportan aún.

Por esto Vue Test Utils llegó como la librería oficial de Vue para hacer testing, así que ahora la pregunta que nos debemos hacer es, ¿qué es lo que hace más fácil Vue Test Utils?

Para responder esa pregunta debemos entender el ciclo de vida de un componente de Vue.

Diagrama de ciclo de vida



Platzi

Herramientas para el testing unitario, ventajas y limitaciones de cada uno

Herramientas

Para realizar pruebas unitarias para una aplicación Vue, el framework provee de un set de herramientas básico y permite elegir entre dos Stacks (conjunto) de bibliotecas distintos para crear las pruebas:

- Vue Test Utils: Un grupo de herramientas provistas por Vue para acceder y manipular componentes y elementos de la aplicación dentro de las pruebas unitarias.
- Jest: Una biblioteca moderna para la definición y ejecución de pruebas unitarias en JavaScript que incluye:
  - Runners
  - Expectativas/Aserciones
  - Biblioteca de Mocking (todo-en-uno).

Pruebas en Vue

Mocha + Chai + Sinon: Un conjunto de bibliotecas para la creación de pruebas unitarias en JavaScript que permiten la:

- Especificación y ejecución (Runner) de pruebas
- Definición de Expectativas/Aserciones
- Implementación de Mocking respectivamente.

Para el caso de Mocha + Chai , Sinon no es instalado automáticamente por Vue, y debe ser instalado manualmente como dependencia de desarrollo, pero es la biblioteca de Mocking que con mayor frecuencia se encontrará asociada a suites de tests basadas en Mocha .

Utilización de objetos simulados (mocks y stubs) En qué consisten y por qué utilizarlos

Mocks

En Sinon, un Mock es una función sustituta como los Spies y Stubs, pero que además de ofrecer comportamientos preprogramados como los Stubs, ofrece expectativas preprogramadas.

El Mock actuará como un sustituto del objeto completo, y combinará los beneficios de funciones Stubs y Spies en la implementación.

Sinon recomienda el uso de Mocks para reemplazar la unidad que está siendo testeada. Se utilizará un Mock entonces cuando se desea controlar como la unidad está siendo utilizada se expresa de una manera, en la que se testea la expectativa de un comportamiento, en vez de realizar una aserción sobre un hecho.

Sinon también indica que como regla general, no debería existir más de un Mock en un test, y si no se desea testear una llamada específica es mejor realizar un Stub.

Pruebas en Vue

La definición de qué es un Mock para Sinon es muy particular, y en general las situaciones

en las que se utilizará son probablemente menos recurrentes y muy específicas.

Un ejemplo de un Mock de Sinon, donde se imita el método "login" de un módulo importado

llamado "Auth", y se pre-establecen las expectativas de que el método login solo sea

llamado una vez y con una lista particular de argumentos

```
it( 'Invoca al servicio de authenticación con el usuario y contraseña'
,
() => {
/* ... */
const authMock = sinon.mock(Auth)
authMock.expects( "login" ).once().withArgs(
"usuario1@misitio.com" ,
"123password" )
/** Resto del codigo del test para invocar un login ... **/
authMock.verify()
authMock.restore()
})
```

Al final del test se verifican las expectativas del Mock y luego se reestablece el módulo a su versión original.

Se observa que en el caso de los tests para Vue con el stack Mocha + Chai , la misma verificación pudo haberse realizado utilizando Stubs y Expectativas de Chai. Se recomienda utilizar Stubs y Spies siempre que sea posible.

Stubs

En Sinon, los Stubs son funciones Spy con comportamientos pre-programados y por lo tanto ofrecen las mismas propiedades y funciones de los Spies, además de las que son propias de los Stubs.

Los stubs pueden ser creados de la misma manera que los Spies, pero a diferencia de los últimos, una llamada a un Stub no ejecutará la función original .

Se recomienda usar Stubs cuando se desea controlar el comportamiento de un test para forzar una situación, o cuando se desea evitar que un método específico sea llamado directamente porque produce un efecto no deseado (realizar una petición HTTP a un servidor remoto, lanzar un proceso lento, de alto consumo de recursos o de comportamiento impredecible)

Pruebas en Vue

La creación de un Stub se puede realizar de cuatro formas distintas:

- var stub = sinon.stub() : Crea un stub de una función anónima.

- var stub = sinon.stub(objeto, "nombreMetodo"): Crea un Stub de un método dentro de un objeto.

- var stub = sinon.stub(objeto, "nombreMetodo").callsFake(fn) : Crea un Stub de un método y reemplaza la implementación del método por la función contenida en la variable fn. Luego se puede establecer con stub.restore() o objeto.metodo.restore() .

- var stub = sinon.stub(objeto): Crea stubs para todos los métodos de un objeto.

Pruebas en Vue

En el siguiente ejemplo realiza el stub de un método de un objeto, Se indica el

comportamiento mediante una función.

Luego se indican distintos Stubs para las primeras 4 llamadas al método y otra opcional

para cuando el primer parámetro con el que se llame el método sea específicamente "Bob

Ross".

```
var stub = sinon.stub(miObjeto, "miMetodo" )
stub.callsFake( (nombre, edad) => {
    return `Te llamas ${nombre} y tienes ${edad} años!`
})
.onFirstCall().resolves( true )
.onSecondCall().throws( "Error Inesperado" )
.onThirdCall().rejects({ data: undefined, error: "Nombre no
  encontrado" })
.onCall(4).returns( 'Te llamas NOMBRE y tienes EDAD años' )
stub.withArgs( "Bob Ross" )
   .returns( "Hola Bob, Cualquier cosa que estés dispuesto a practicar
la podrás realizar!" )
```

Pruebas end-to-end

Competencias

- Construir tests unitarios de estados para una aplicación que utiliza Vuex.
- Construir una suite de tests end-to-end utilizando Cypress o Nightwatch para realizar pruebas en una aplicación web.

Introducción

Los test E2E , End-to-end o en español: "de principio a fin" , al contrario de los tests unitarios, son un tipo de pruebas automatizado que permite testear un sistema en una situación en la que varios componentes están involucrados. La especificación que se desea comprobar es una regla de negocio, una funcionalidad del sistema o un caso de uso.

Es decir, en vez de testear el comportamiento discreto de un único componente un test E2E testea un comportamiento que involucra la participación de varios componentes de manera integrada.

En este capítulo se explora cómo crear pruebas end-to-end para aplicaciones Vue, cómo identificar casos de pruebas y qué herramientas se pueden utilizar para implementar los tests.

Tests end-to-end en Vue

Al crear una aplicación Vue con Vue CLI v3, y seleccionar las características del proyecto manualmente, con el mismo procedimiento descrito anteriormente, es posible seleccionar un Stack para la implementación de pruebas E2E.

Cuando la característica de pruebas E2E es seleccionada, el menú de opciones consultará en algún momento si se desea utilizar como Runner de pruebas E2E los frameworks Cypress o Nightwatch .

Pruebas en Vue

Características de las pruebas end to end, ventajas y limitaciones, diferencia con pruebas unitarias

El objetivo de cada test E2E es comprobar las funcionalidades que ofrece el sistema, es decir, las características (Features), o requerimientos funcionales.

Estas se pueden encontrar en:

- La lista de funcionalidades que fueron identificadas ("Levantadas") al inicio de un proyecto, entre el cliente, desarrolladores, dueño de producto, etc.
- Los casos de uso (que probablemente involucran interacción del usuario) que fueron identificados durante una etapa de diseño.
- Historias de usuario, escenarios o flujos recogidos durante la conversación con el cliente, por alguna unidad de diseño y experiencia de usuario, etc.

diferencia de las pruebas unitarias, no se necesita una biblioteca especial de Vue para interactuar con los elementos dentro del navegador.

Sin embargo al seleccionar un framework, Vue instalará una biblioteca de funciones que permita integrar los comandos del test Runner, los de compilación de los códigos fuentes (Webpack bundling) y levantamiento de un servidor web, así como la configuración inicial.

Pruebas en Vue

Por ejemplo, Vue instalará @vue/cli-plugin-e2e-nightwatch en el caso de Nightwatch, y así utilizar vue-cli-service dentro del NPM script test:e2e para lanzar las pruebas con Nightwatch.

Cuando desde la línea de comandos se escriba lo siguiente:

npm run test :e2e

Npm a través de vue-cli-service , se encargará de recompilar el bundle con webpack, levantar un servidor de desarrollo, y llamar al test runner , indicando incluso la URL base desde donde

dirigir el navegador para empezar la ejecución de los tests.

Setup de herramientas con vue-cli

Al crear una aplicación Vue con Vue CLI v3, y seleccionar las características del proyecto manualmente, con el mismo procedimiento descrito anteriormente, es posible seleccionar
un Stack para la implementación de pruebas E2E.
Cuando la característica de pruebas E2E es seleccionada, el menú de opciones consultará
en algún momento si se desea utilizar como Runner de pruebas E2E los frameworks Cypress o Nightwatch .
Imagen

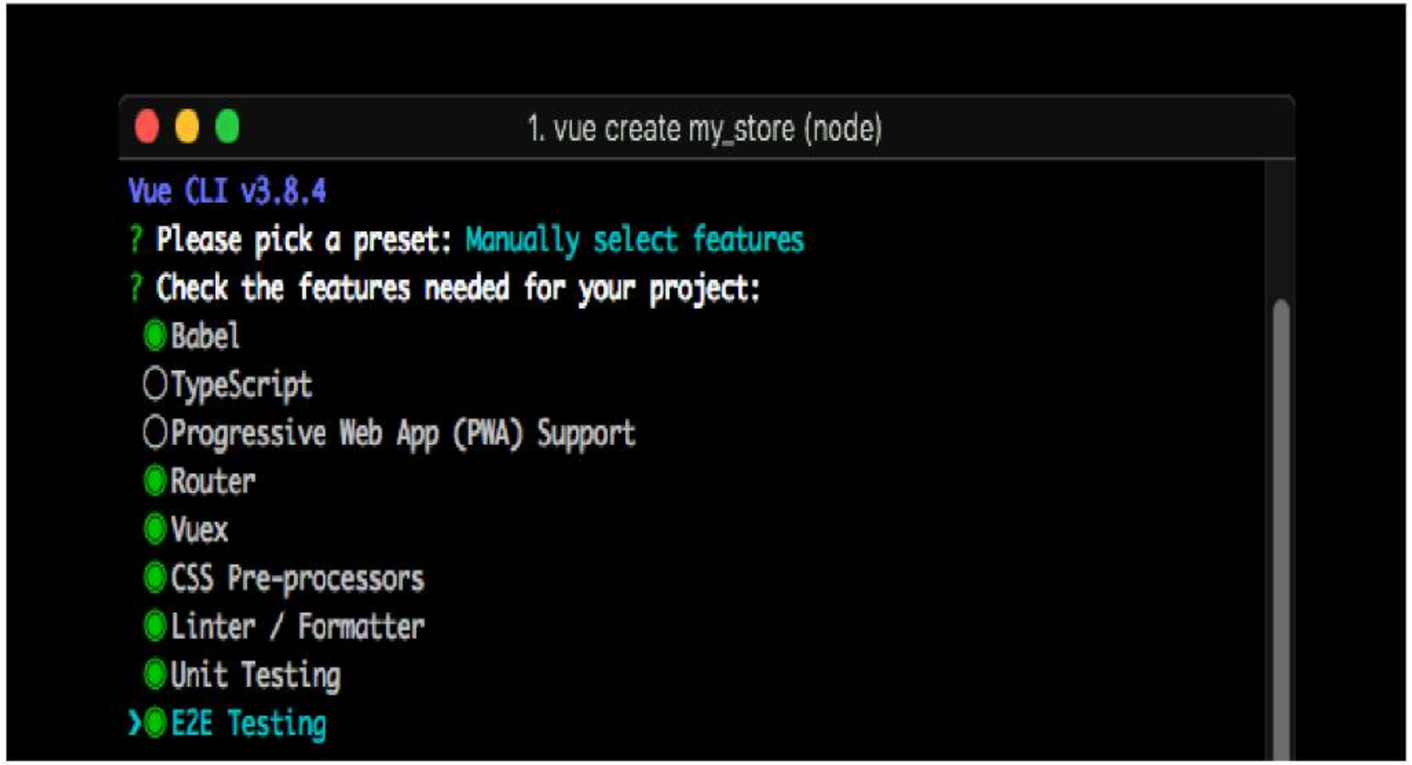


Imagen 1. Opciones de características para la creación del nuevo proyecto Vue

Pruebas en Vue

Images 2. Opciones de runners para pruebas E2E.

En las imágenes de las figuras Imagen 1 y Imagen 2 se muestra el momento en que se selecciona la característica de pruebas E2E y posteriormente el menú que permite elegir el framework a utilizar para implementar las pruebas.

Para cualquiera de las opciones, como es costumbre, se generará un test de ejemplo correspondiente al framework seleccionado. Esta vez en el directorio . /tests/e2e/ .

Herramientas para el testing end-to-end

A diferencia de las pruebas unitarias, no se necesita una biblioteca especial de Vue para interactuar con los elementos dentro del navegador.
Sin embargo al seleccionar un framework, Vue instalará una biblioteca de funciones que permita integrar los comandos del test Runner, los de compilación de los códigos fuentes (Webpack bundling) y levantamiento de un servidor web, así como la configuración inicial.
Por ejemplo, Vue instalará @vue/cli-plugin-e2e-nightwatch en el caso de Nightwatch, y así utilizar vue-cli-service dentro del NPM script test:e2e para lanzar las pruebas con Nightwatch.
Cuando desde la línea de comandos se escriba lo siguiente:

```
npm run test :e2e
```

Npm a través de vue-cli-service , se encargará de recompilar el bundle con webpack, levantar un servidor de desarrollo, y llamar al test runner , indicando incluso la URL base desde donde dirigir el navegador para empezar la ejecución de los tests.

Cypress

Cypress es un framework de pruebas end-to-end Open Source moderno, que entre sus características más novedosas permite una fácil integración con servicios de integración continua (CI), y facilita la depuración de tests cuando ocurren errores.

Mientras el Test Runner es open source, Cypress.io ofrece un servicio de pago con un dashboard para monitoreo y administración de pruebas sobre CI.

El Test Runner es una aplicación de Electron (Chromium) con una interfaz gráfica para administrar y observar el resultado de los tests. y es posible instalar una gran variedad de "Plugins" para facilitar el desarrollo de las pruebas, interactuar con servicios de terceros, etc.

Las pruebas se ejecutarán en la instancia de Chromium o en el navegador Chrome si se encuentra instalado.

Nightwatch

Nightwatch permite automatizar pruebas para aplicaciones web, mediante el uso de la API W3C WebDriver , antiguamente conocida como "Selenium WebDriver" . Una tecnología orientada a estandarizar la automatización en navegadores web.

La automatización en Nightwatch se realiza internamente mediante la comunicación con un servicio que recibe comandos a través de una API REST y hasta el navegador mediante la WebDriver API. El servicio intermediario es llamado Selenium Server y requiere de la instalación de Java versión 7 o más reciente .

–

Setup de herramientas con vue-cli

Al crear una aplicación Vue con Vue CLI v3, y seleccionar las características del proyecto manualmente, con el mismo procedimiento descrito anteriormente, es posible seleccionar
un Stack para la implementación de pruebas E2E.
Cuando la característica de pruebas E2E es seleccionada, el menú de opciones consultará
en algún momento si se desea utilizar como Runner de pruebas E2E los frameworks Cypress o Nightwatch .
Imagen

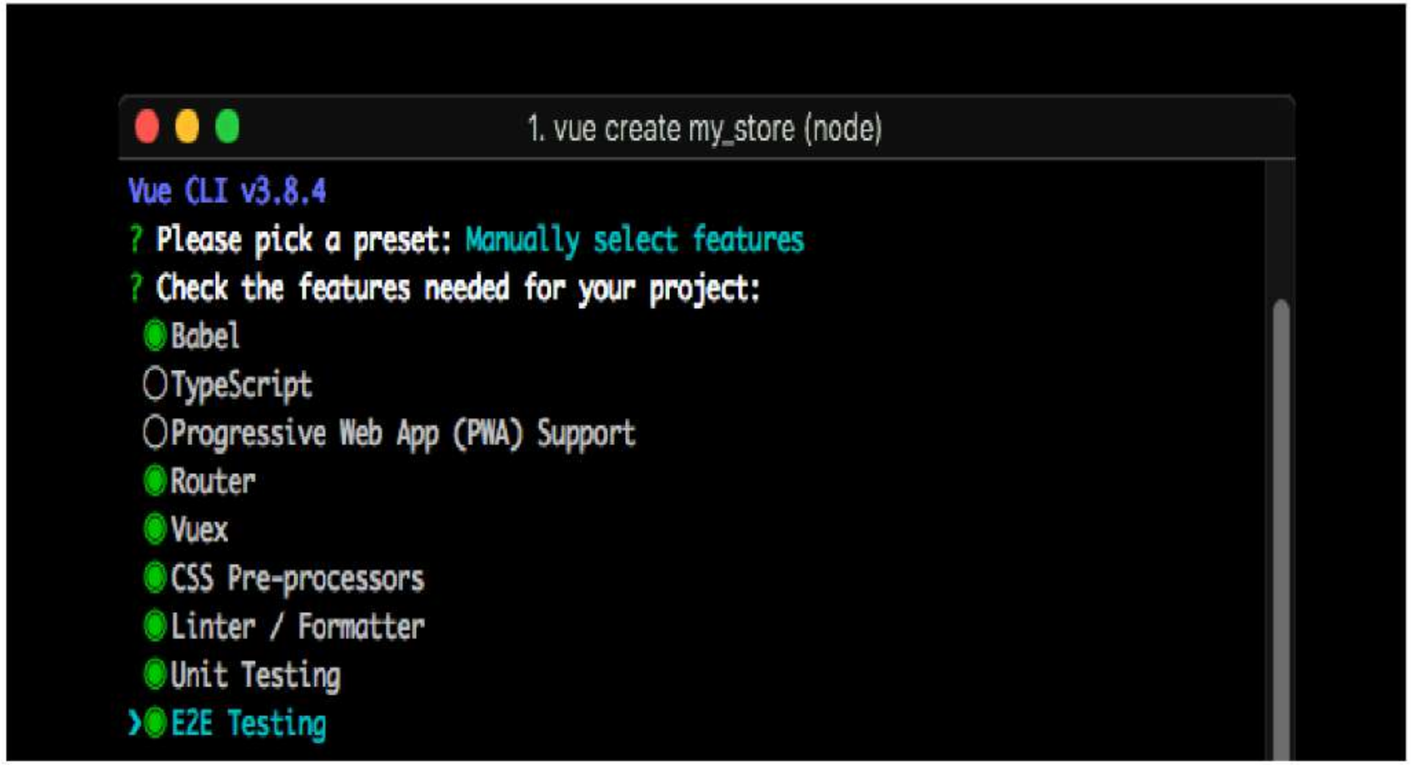


Imagen 1. Opciones de características para la creación del nuevo proyecto Vue

sustantiva

CON • SENTIDO